Technical Documentation

Developer and system reference

CSCI 441 – Software Engineering

Team AO3 | Demo 1 | November 9, 2026

Team members: Yean Fong, Phallamonunhh Chun, Bunkheang Heng, Heejae Choi, Sereyprophea Sarin, Doungkhe Neang

Project: SuperAsk.AI – AI-Powered Government Service Assistant

Codebase reviewed: Super-ask repository ZIP provided for Demo 1.

1. Purpose and scope

This document explains how to run, understand, test, and maintain the SuperAsk.AI Phase 0 prototype. It is written for a developer or technical reviewer using the project repository. The application is a Next.js web service with a shared question-answer engine used by the web interface and Telegram.

Important: This is a course prototype. The repository explicitly states that its knowledge base is sample/experimental and not formally approved by government content stewards. Do not treat the system as an authoritative government service or treat sample fees and deadlines as official.

2. Repository and technology stack

Part	Implementation
Frontend	Next.js 15, React 19, TypeScript, Tailwind CSS 4
Backend/API	Next.js App Router handlers, Node.js runtime
Answer engine	Policy guardrails, curated-answer matching, retrieval, confidence checks, generation, claim verification and citation assembly
Model providers	SEA-LION, Anthropic, vLLM or extractive fallback, selected by environment settings
Messaging	Telegram webhook and Telegram Mini App entry
Testing	Vitest, V8 coverage, TypeScript type checking, evaluation scripts and GitHub Actions
Hosting	Vercel is referenced as the prototype deployment

Major source directories

Path	Purpose
app/	Landing page, chat page, login/register prototype and API endpoints
components/chat/	Question composer, conversation display, citations, sidebar, feedback
lib/engine/	Answer selection, guardrails and verification
lib/retrieval/	Search and ranking of existing knowledge-base passages
lib/llm/	Provider abstraction and fallback implementation
lib/telegram/	Telegram formatting and bot integration
lib/kb/ and data/	Data loading, knowledge base, glossary, aliases and source registry
lib/crawl/ and scripts/	Source monitoring and maintenance utilities
tests/	Automated tests; separate integration materials may be kept elsewhere in the repository
slides/ and documentation/	Demo materials and submission PDFs, if present in the updated repository

3. Prerequisites and local installation

• Install Node.js and npm compatible with this project. The codebase and deployment documentation refer to Node.js 24.

• Download or clone the repository and open the project root, where package.json is located.

```
npm install
```

• Copy the supplied environment template to a local environment file. On Windows PowerShell:

```
Copy-Item .env.example .env.local
```

• On macOS/Linux, use:

```
cp .env.example .env.local
```

• Start the development server and open http://localhost:3000 in your browser:

```
npm run dev
```

No external AI credential is required for the basic prototype flow: when no usable model credential is configured, the application can use its deterministic extractive fallback. This is useful for a first local test, although results may differ from a deployed model-backed environment.

Environment variables and secrets

Edit **.env.local** locally if model or Telegram integration is needed. The template documents LLM_PROVIDER (sealion, anthropic, vllm or extractive), relevant model keys/URLs, retrieval thresholds, and Telegram variables including TELEGRAM_BOT_TOKEN, TELEGRAM_WEBHOOK_SECRET, TELEGRAM_MINI_APP_URL and NEXT_PUBLIC_SITE_URL. Never upload actual keys or the populated .env.local file to GitHub.

For a server deployment, provide variables using the hosting platform’s secret-management interface; do not expose API keys to browser-side JavaScript.

4. Architecture and request processing

The web chat sends a JSON request to POST /api/ask. The route validates the question (required and at most 1,000 characters), limits supplied history, calls the shared answer engine, records an audit event, and returns a JSON response. If processing fails unexpectedly, the endpoint returns a safe decline rather than an unsupported answer.

Question processing sequence used in the Demo 1 slides: (1) policy screen, (2) curated answer match, (3) passage retrieval, (4) confidence gate, (5) model generation when appropriate, (6) claim verification, (7) output moderation, and (8) citation assembly. The system is designed to decline when information is off-topic or inadequately supported.

The knowledge base is separate from monitored websites: the crawler may record a changed page, but fetched text does not automatically become answerable content. A content-approval portal is not yet implemented.

5. APIs and integrations

Route	Purpose / notes
POST /api/ask	Process a citizen question and return a sourced answer or decline.
POST /api/feedback	Receive feedback about an answer.
GET /api/health	Health endpoint used for deployment checks.
POST /api/telegram/webhook	Receive Telegram updates using secret validation and route them through the shared engine.

For the Telegram webhook, the repository includes **scripts/telegram-webhook.ts**. The environment template provides an example registration command:

```
npx tsx scripts/telegram-webhook.ts set https://<host>/api/telegram/webhook
```

Only register a webhook after configuring the bot token and webhook secret in a trusted environment. The Mini App opens the chat experience inside Telegram.

6. Knowledge base and source management

The repository’s data/README.md identifies 45 source passages, five curated answers, a terminology glossary and search aliases. Data comes from several government-service domains, including transport, civil registration, trademarks and consumer finance. Some content is deliberately marked as sample or not steward-approved.

• data/kb/: passages used for retrieval, including provenance, source URLs and dates.

• data/curated.json: curated response entries; data/glossary.json and data/aliases.json: language assistance.

• data/sources.json: generated registry of 132 monitored entries, not a list of 132 answerable websites.

• data/crawl/: monitoring output; not automatically published into the answerable knowledge base.

The source registry is generated from upstream registry data. Run **npm run build-sources** to regenerate it; do not manually edit generated sources.json.

7. Testing and validation

The codebase provides the following commands. Run them from the repository root:

Command	Purpose
npm test	Run the configured Vitest suite.
npm run test:coverage	Run tests with V8 coverage reporting.
npm run typecheck	Check TypeScript types.
npm run build	Build the production application.
npm run eval	Run the retrieval/refusal evaluation harness.
npm run check	Run typecheck, tests and evaluation/quality scripts.

Command	Purpose
npm run crawl:verify	Verify reachability of configured sources.
npm run crawl:monitor	Run content monitoring.

The presentation reports 610 unit tests and 33 integration tests (643 total), with 89.6% line coverage. The uploaded ZIP’s README refers to a different test count, indicating the repository and slides may represent different revisions. These numeric results are presentation claims and should be verified against a fresh test run before being repeated as current results.

Additional integration tests uploaded in a separate directory may require their own Vitest configuration. Refer to their README3.txt and re-check relative paths after relocating the directory.

8. Deployment, maintenance and limitations

• For production-like deployment, set environment variables on the host, build the project and verify GET /api/health and both web and Telegram entry points.

• Keep credentials in server-side settings and review CI failures before merging changes.

• Re-run tests and evaluation after modifying the answer engine, glossary, retrieval rules or sources.

• Treat reported information as guidance: encourage visitors to check linked official sources before relying on procedures, fees or deadlines.

• Known Phase 0 limits include prototype-only account screens, no completed steward portal, a limited unapproved/sample corpus, lightweight ranking alternatives and less reliable Telegram follow-up memory across serverless restarts.

Repository reference: https://github.com/Monunhh/Super-ask